

ChargeGrid Intelligence

Documentacao Tecnica Sprint 03

1. Continuidade da Sprint 02

A Sprint 02 introduziu múltiplas sessões, controle de demanda (limite de 30 kW), tarifação dinâmica e integração simulada OCPP/MODBUS. O usuário interagia através de um menu básico.

A Sprint 03 dá continuidade direta a esse código. As regras de negócio (tarifas, limite de potência e tipos de veículos) são mantidas, mas a **estrutura interna do código foi refatorada** para utilizar Classes (Orientação a Objetos) em conjunto com listas, além de funções (def), tratamento de erros e algoritmos manuais de ordenação e busca.

Aspecto	Sprint 02	Sprint 03
Estrutura de Dados	Dicionários ou listas simples	Instâncias de uma Classe (Orientação a Objetos)
Identificação	Índice sequencial implícito	ID Único (obrigatório, numérico, sem repetição)
Ordenação	Ordem de inserção	Algoritmo Bubble Sort pelo ID da sessão
Busca de Registros	Não possuía	Busca Binária (requer ordenação prévia)
Relatório Consolidado	Energia total e Valor total	Inclusão de Ticket Médio, Maior e Menor consumo

2. Objetivo

Refatorar o simulador para representar sessões como instâncias de uma classe armazenadas em uma lista, prevenir quebras de código via try/except, implementar busca e ordenação manuais e gerar estatísticas com análise de complexidade (Big-O).

3. Menu interativo

- | | |
|-----------------------------------|-------------------------------------|
| 1 - Criar sessao | 6 - Buscar Sessão (Busca Binária) |
| 2 - Simular sessões desordenadas | 7 - Gerar relatório de estatísticas |
| 3 - Ordenar Sessões (Bubble Sort) | 8 - Alternar horario de pico |
| 4 - Listar sessoes | 9 - Sair |
| 5 - Processar recargas | |

4. Representação da Estrutura de Dados (Classes e Listas)

Para representar uma sessão de recarga e armazenar seus dados, optamos por utilizar o conceito de **Classes** (Orientação a Objetos). A escolha por Classes permite centralizar os atributos (como id, nome, potencia, horas, custo, etc.) e inicializá-los facilmente via método construtor `__init__`.

Para registrar e manipular múltiplas sessões, declaramos uma lista vazia no início do programa (`sessoes = []`). Quando uma nova recarga é informada, o sistema instancia a classe Sessao e a adiciona à lista principal usando `sessoes.append(nova_sessao)`.

5. Tratamento de Dados e Validações

6. Algoritmo de Ordenação

A ordenação das sessões foi implementada manualmente sem o uso da função `sort()`. Utilizamos o algoritmo **Bubble Sort** para ordenar a lista de sessões pelo ID.

Funcionamento: O algoritmo captura o tamanho da lista com `len(sessoes)` e aplica dois laços de repetição. O laço interno compara os IDs adjacentes (ex: `if sessoes[j].id > sessoes[j+1].id`). Se estiverem fora de ordem, os objetos trocam de posição. Isso garante que o objeto com o maior ID seja levado para o final da fila repetidas vezes, até ordenar tudo.

7. Algoritmo de Busca

Implementamos a **Busca Binária** para localizar rapidamente uma sessão digitando o seu ID.

Funcionamento: Como a busca binária exige que os dados estejam classificados, a função aciona a ordenação previamente. O código então acessa o valor do meio da lista. Se o ID for menor que o do meio, a busca passa a ignorar a metade superior; se for maior, ignora a inferior. O processo divide o tamanho da lista pela metade sucessivamente até o ID bater com o pesquisado.

8. Análise de Complexidade (Notação Big-O)

- **Bubble Sort — $O(n^2)$:** Possui complexidade de tempo quadrática devido aos dois laços aninhados no código. Na prática, se a quantidade de recargas na lista principal dobrar, o número de comparações e o tempo de processamento aumentarão cerca de quatro vezes. É ineficiente para um volume gigantesco de dados.
- **Busca Binária — $O(\log n)$:** Possui complexidade de tempo logarítmica devido à divisão contínua da área de busca pela metade. Na prática, se a lista dobrar de tamanho, o algoritmo precisará de apenas uma verificação a mais para encontrar a resposta. É incrivelmente rápido e eficiente.

9. Estatísticas da Estação

A função de relatório utiliza um `for` para iterar sobre todos os objetos cadastrados, calculando em tempo real os indicadores da estação:

- **Quantidade de sessões:** Obtida utilizando o `len(sessoes)`.
- **Energia fornecida e Faturamento:** Soma dos atributos de consumo (`s.energia`) e custo (`s.custo`) nas instâncias processadas.
- **Ticket Médio:** Faturamento total dividido apenas pelas sessões contabilizadas com o status de "Finalizada".
- **Maior e Menor consumo:** Utiliza variáveis comparativas (iniciando o menor consumo com infinito) para isolar os picos máximo e mínimo de kW gerados no pátio.

10. Conclusão

O simulador da Sprint 02 foi totalmente adequado para a Sprint 03. Mantendo as regras de negócio de energia, o sistema agora gerencia dados de forma madura usando Classes e Listas. Previne o encerramento por erros de usuário, usa funções modulares e aplica algoritmos clássicos (Busca Binária e Bubble Sort) para manipular os registros, gerando análises estatísticas reais da estação.